Phase 2

RISC-V RV32I Module Design in Verilog

EEL 4768 Computer Architecture
Fall 2026

Learning Objectives

By the end of this phase you should be able to:

- Write structural and dataflow Verilog for both combinational and sequential logic.
- Build the four initial blocks detailed in this phase.
- Debug your own hardware designs to fully test them, considering all possible inputs.

Note

You must write your own Verilog test benches to verify your designs. An example test bench will be provided, as well as skeletons for each of the modules.
You also should reference the RV32I reference card from webcourses for the instruction formats, opcodes, and immediate values.

1 Submission

- **One submission per group.** You will manually submit your work to Webcourses as a zip file. Students in multiple groups, or not in a group by the time of submission, will receive **no credit** for the submission.
- **Files to submit**:

```
alu.v
imm.v
rf.v
decoder.v
```

  One module per file, and name the module the same as the filename. The testbenches instantiate modules named `alu`, `imm`, `rf` and `decoder` by name.

Note

Skeletons for each verilog file can be found in `phase_2/skeletons/`. You must use these skeletons, or the grader will reject your submission.

2 Grading Rubric

Category	Testbench	Points
ALU	`alu_tb.v`	0.75
Immediate Generator	`imm_tb.v`	0.75
Register File	`rf.v (no bypass)`	0.25
	`rf.v (with bypass)`	0.5
Instruction Decoder	`decoder_tb.v`	0.75
Total		**3.0**

3 Verilog Coding Rules

Your submission is internally checked against a synthesizable subset. This is to insure your code will be synthasizable for future phases. The rules listed below will result in a 0 for the specific file.

Rule	What it means for you
No `*`, `/`, `%`, `**`	Arithmetic that does not synthesize cheaply. You do not need any of them in this phase.
No `===`, `!==`, `==?`, `!=?`	Not synthesizable. Use `==` and `!=`.
Shifts only by a constant	`x << 3` is fine; `x << i_op2[4:0]` is *rejected*. See Section **??**.
No `if`/`else` in combinational logic	`if`/`else` is only legal inside a clocked `always @(posedge ...)` block. For combinational logic use a continuous `assign` with the `?:` operator, or a `case` inside `always @(*)`.
No procedural loops	`for`, `while`, `repeat` and `do`/`while` statements are rejected. To build something repetitive, use a `generate` loop, or just write the instances out.
No delays or waits	`#5`, `wait`, `disable` and friends are simulation constructs.
No system tasks	`$display`, `$finish`, `$stop`, `$system`, `$readmemh`/`$readmemb`, `$writememh`/`$writememb` and all file I/O (`$fopen`, `$fwrite`, ...) are not allowed in your final submission. Only constant functions such as `$clog2` are allowed.
No `` `include ``	Not needed for importing other files.

4 Part 1: ALU

For the first part, you will be tasked with designing a 32-bit ALU following the RISC-V instruction set. This is a purely combinational ALU. Each ALU operation should be implemented manually. You cannot use +, -, <<, >>, or any other arithmetic operations inplace of the ALU operations. Follow the table below for how to implement each operation.

Operation	**Implementation**
Add / Sub	2's compliment Lookahead Carry Adder
SLL, SRL, SRA	Barrel Shifter
SLT / SLTU	Comparator
XOR, OR, AND	Bitwise Logic

Below lists the input and output signals, and what they represent.

Port	**Dir/Width**	**Meaning**
i_opsel	in [2:0]	Selects the operation (table below).
i_sub	in	With i_opsel = 3'b000, subtract instead of add. Ignored for every other i_opsel.
i_unsigned	in	Comparisons are unsigned. Only used for branch comparisons. Unsigned arithmetic does not use this.
i_arith	in	With i_opsel = 3'b101, shift right *arithmetically* (replicate the sign bit) instead of logically.
i_op1	in [31:0]	First operand. The value shifted, for shifts.
i_op2	in [31:0]	Second operand. **For shifts, only i_op2[4:0] is the shift amount**; the upper 27 bits are ignored.
o_result	out [31:0]	The operation's result.
o_eq	out	i_op1 == i_op2.
o_slt	out	i_op1 < i_op2, signed or unsigned per i_unsigned.

i_opsel	**Operation**	o_result
3'b000	ADD / SUB	i_op1 + i_op2, or i_op1 - i_op2 when i_sub.
3'b001	SLL	i_op1 shifted left by i_op2[4:0].
3'b010	SLT	i_op1 < i_op2, signed.
3'b011	SLTU	i_op1 < i_op2, unsigned.
3'b100	XOR	i_op1 ^ i_op2.
3'b101	SRL / SRA	i_op1 shifted right by i_op2[4:0], logically or (when i_arith) arithmetically.
3'b110	OR	i_op1 | i_op2.
3'b111	AND	i_op1 & i_op2.

Clarification

o_eq and o_slt are always checked. Even if they are not directly used, it is important to always insure they output.

5 Part 2: Immediate Generator

Part 2 implements an immediate generator. This is acombinational block that extracts and sign-extends the immediate value from an 32-bit instruction word.

Port	Dir/Width	Meaning
`i_inst`	in [31:0]	The full 32-bit instruction word.
`i_format`	in [5:0]	One-hot Selector for the instruction format.
`o_immediate`	out [31:0]	The 32-bit sign-extended immediate.

`i_format`	Bit	Format
`6'b000001`	[0]	R
`6'b000010`	[1]	I
`6'b000100`	[2]	S
`6'b001000`	[3]	B
`6'b010000`	[4]	U
`6'b100000`	[5]	J

Table 1: The six instruction formats and the one-hot `i_format` code that selects each.

6 Part 3: Register File

This part implements a 32-bit register file, or the registers of your processor. Reads are combinational, while writes are sequential.

Port	Dir/Width	Meaning
`i_clk`	in	Clock. Everything sequential happens on its rising edge.
`i_rst`	in	**Synchronous**, active high. Clears all 32 registers to zero.
`i_rs1_raddr`	in [4:0]	Read port 1 address.
`o_rs1_rdata`	out [31:0]	Read port 1 data.
`i_rs2_raddr`	in [4:0]	Read port 2 address.
`o_rs2_rdata`	out [31:0]	Read port 2 data.
`i_rd_wen`	in	Write enable.
`i_rd_waddr`	in [4:0]	Write address.
`i_rd_wdata`	in [31:0]	Write data.

The module takes **one parameter**, the bypass enable, and it must be the *first* parameter — the two testbenches instantiate it positionally, as `rf #(0)` and `rf #(1)`:

```
module rf #(parameter BYPASS_EN = 0) (
    input  wire        i_clk,
    /* ... */
);
```

The bypass enable allows the register file to enact read-before-write behavior, so that a read port can see the new value of a register on the same clock edge it is written. You have to handle both cases for full points.

6.1 Behavior

- `x0` **is hardwired to zero.** A write to address `5'b00000` must be discarded, and a read of address `5'b00000` must return `32'b0` forever after. Both testbenches write `32'hDEADBEEF` to `x0` as their very first check.
- **Writes are synchronous**, taking effect on the rising edge of `i_clk` when `i_rd_wen` is high.
- **Reads are combinational.** `o_rs1_rdata` tracks `i_rs1_raddr` with no clock edge in between.
- **Reset is synchronous and active high**, applied inside the same clocked block as the write.
- **Bypass**, when `BYPASS_EN` is nonzero: if a write is in flight (`i_rd_wen` is high) to the same register a read port is addressing, and that register is not `x0`, that read port returns `i_rd_wdata` *immediately*, without waiting for the clock edge. When `BYPASS_EN` is zero the read port returns the stored value, and the write is only visible after the edge.

7 Part 4: Instruction Decoder

The final part has you constructing the instruction decoder. This is a combinational block that takes one 32-bit instruction word and produces every control signal the datapath needs.

Note

The decoder instantiates uses generator you wrote in Section **??**. You must instantiate the imm module within your decoder.

7.1 Instruction fields

Everything the decoder does is a function of these fields:

Field	Bits	Use
`opcode`	`i_inst[6:0]`	Selects the instruction class.
`rd`	`i_inst[11:7]`	Destination register specifier.
`funct3`	`i_inst[14:12]`	Sub-operation within a class.
`rs1`	`i_inst[19:15]`	First source register specifier.
`rs2`	`i_inst[24:20]`	Second source register specifier.
`funct7`	`i_inst[31:25]`	Further sub-operation (add vs. sub, srl vs. sra).

7.2 Outputs

Legality and halt..

Port	Width	Meaning
`o_legal`	1	The word is a legal RV32I encoding.
`o_halt`	1	The instruction is `ebreak` (`32'h00100073`).

Note

`ecall` is *not* legal in this decoder. `ebreak` (`32'h00100073`) is the only SYSTEM instruction, and it is what drives `o_halt`. `ecall` (`32'h00000073`) is tested in the ILLEGAL group and must produce `o_legal = 0` and `o_halt = 0`. This differs from Phase 1, where programs terminate with `ecall`.

Register specifiers and immediate.

Port	Width	Meaning
`o_rs1`	[4:0]	`i_inst[19:15]`, for instructions that read `rs1`.
`o_rs2`	[4:0]	`i_inst[24:20]`, for instructions that read `rs2` (OP, STORE, BRANCH).
`o_rd`	[4:0]	`i_inst[11:7]` for instructions that write a register, and `5'd0` **for every instruction that does not** — stores, branches, and any illegal word.
`o_immediate`	[31:0]	The immediate for this instruction's format (Table **??** lists the format codes). Instantiate your `imm` module here.

ALU control.

Port	Width	Meaning
o_op1_sel	1	ALU operand 1 source: 0 = rs1, 1 = PC. Only auipc selects the PC.
o_op2_sel	1	ALU operand 2 source: 0 = rs2, 1 = immediate. Set for OP-IMM, LOAD, STORE, auipc and jalr.
o_alu_opsel	[2:0]	alu's i_opsel: funct3 for OP and OP-IMM, and 3'b000 (add) for everything else that uses the ALU.
o_alu_sub	1	alu's i_sub. Set *only* for sub (OP, funct3 000, funct7 0100000); clear for every address calculation.
o_alu_unsigned	1	alu's i_unsigned. Set for sltu/sltiu, and equal to funct3[1] for branches (so bltu/bgeu set it).
o_alu_arith	1	alu's i_arith, funct7 == 7'b0100000, for sra/srai.

Branch and jump control.

Port	Width	Meaning
o_branch	1	This is a (legal) conditional branch.
o_jump	1	This is jal or jalr.
o_branch_equal	1	The branch tests equality (beq, bne) rather than ordering: ~funct3[2].
o_branch_unsigned	1	The ordering comparison is unsigned (bltu, bgeu): funct3[1].
o_branch_invert	1	Take the branch on the *negation* of the test (bne, bge, bgeu): funct3[0].
o_pc_sel	1	Target base: 0 = PC (branches, jal), 1 = rs1 (jalr).

Data-memory control.

Port	Width	Meaning
o_dmem_ren	1	This is a load.
o_dmem_wen	1	This is a store.
o_dmem_align	[1:0]	Alignment mask of the access: 2'b00 byte, 2'b01 half-word, 2'b11 word.
o_dmem_memb	1	Byte access (lb, lbu, sb).
o_dmem_memh	1	Half-word access (lh, lhu, sh).
o_dmem_memw	1	Word access (lw, sw).
o_dmem_memu	1	The loaded value is *zero*-extended rather than sign-extended (lbu, lhu).

Writeback select. o_rd_sel [3:0] is **one-hot** and chooses what gets written to rd:

Bit	Value	Source	Instructions
[0]	4'b0001	ALU result	OP, OP-IMM, auipc
[1]	4'b0010	immediate	lui
[2]	4'b0100	PC + 4	jal, jalr
[3]	4'b1000	memory	LOAD

8 Testing Your Work

There is no autograder in this repository. Verifying that `alu.v`, `imm.v`, `rf.v` and `decoder.v` behave correctly is part of the assignment. There are provided trace files that you can use to validate your designs, but must write the test harnesses yourself. The provided trace files can be found in `phase_2/traces/`. For the provided register file traces, you must supply your own clock signal.

8.1 The example

`phase_2/example/` holds two files:

- `opmux.v` — a small combinational module with four inputs (`i_a`, `i_b`, `i_sel`, `i_en`) and two outputs (`o_result`, `o_zero`). A two-bit select mux chooses between `+`, `-`, `<<` and `>>`.
- `opmux_tb.v` — a self-checking testbench for it. It is commented as a walkthrough to help you build your own test benches.

Run it with `./phase_2/run_example.sh`, which prints the two commands it uses so you can run them on your own modules:

```
$ iverilog -s opmux_tb -o build/sim example/opmux_tb.v example/opmux.v
$ ./build/sim

========== opmux testbench ==========
--- add ---
[PASS] add: 7 + 9
[PASS] add: 0 + 0 sets zero
...
212 passed, 0 failed
ALL TESTS PASSED
```

`-s` names the top module to elaborate, `-o` names the simulator to write, and every source file the design needs is listed after them.

Note

An expected value copied out of your design proves only that the design agrees with itself. Make sure to fully test all edge cases of your own design.